

Tomcat JDBC Connection Pooling is a high-performance, asynchronous connection pool module replacement for the older Commons DBCP package. It manages a reusable set of database connections, avoiding the performance hit of creating and tearing down connections on every request.

Core Configuration (context.xml)

To set up a global or application-level connection pool, define a <Resource> inside Tomcat’s conf/context.xml or your application's META-INF/context.xml:

```
XML
<Context>
  <Resource name="jdbc/TestDB"
    auth="Container"
    type="javax.sql.DataSource"
    factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"
    driverClassName="com.mysql.cj.jdbc.Driver"
    url="jdbc:mysql://localhost:3306/mydatabase?useSSL=false"
    username="dbuser"
    password="dbpassword"

    <!-- Pool Size Settings -->
    initialSize="5"
    maxActive="50"
    maxIdle="15"
    minIdle="5"
    maxWait="10000"

    <!-- Connection Validation & Health -->
    testOnBorrow="true"
    validationQuery="SELECT 1"
    validationInterval="30000"

    <!-- Connection Leak Prevention -->
    removeAbandoned="true"
    removeAbandonedTimeout="60"
    logAbandoned="true" />
</Context>
```

Essential Pool Properties

Parameter	Recommended Standard	Description
initialSize	5–10	Number of connections established when the pool starts.
maxActive	20–100	Maximum number of active connections allowed simultaneously.
maxIdle	10–20	Maximum number of unused connections kept idle in the pool.

Parameter	Recommended Standard	Description
minIdle	5	Minimum number of idle connections maintained at all times.
maxWait	10000 (10s)	Max wait time (ms) for a connection before throwing an exception.
validationQuery	SELECT 1	Lightweight query used to check if a connection is alive before using it.
testOnBorrow	true	Validates connection before giving it to the application.
validationInterval	30000 (30s)	Prevents validation spam by caching successful checks for <i>X</i> ms.

How to Retrieve Connections in Java

Using JNDI Lookup (Standard Servlet/Jakarta EE approach):

Java

```
import javax.naming.Context;
import javax.naming.InitialContext;
import javax.sql.DataSource;
import java.sql.Connection;
```

```
public class DatabaseUtils {
    public static Connection getConnection() throws Exception {
        Context initContext = new InitialContext();
        Context envContext = (Context) initContext.lookup("java:comp/env");
        DataSource ds = (DataSource) envContext.lookup("jdbc/TestDB");
        return ds.getConnection(); // Borrows connection from pool
    }
}
```

Using Programmatic Configuration (Standalone or Frameworks):

Java

```
import org.apache.tomcat.jdbc.pool.DataSource;
import org.apache.tomcat.jdbc.pool.PoolProperties;
```

```
PoolProperties p = new PoolProperties();
p.setUrl("jdbc:mysql://localhost:3306/mydatabase");
p.setDriverClassName("com.mysql.cj.jdbc.Driver");
p.setUsername("dbuser");
p.setPassword("dbpassword");
p.setMaxActive(50);
p.setInitialSize(5);
```

```
DataSource dataSource = new DataSource();
dataSource.setPoolProperties(p);
```

```
// Retrieve connection
```

```
Connection conn = dataSource.getConnection();
```

Key Advantages

- **High Performance:** Operates with low overhead and handles high-concurrency environments efficiently.
- **Connection Leak Detection:** Automatically reclaims connections if an application opens them without closing (`removeAbandoned="true"`).
- **Asynchronous Retrieval:** Supports returning `Future<Connection>` directly for async execution flows.